

Laboratorio 3 — Algoritmos de Enrutamiento

CC3067 Redes — Universidad del Valle de Guatemala

Jose Sanchez 231221

Ángel Mérida 23661

Eliazar Canastuj 23384

1. Descripción de los Algoritmos Utilizados y su Implementación

Se implementaron los cuatro algoritmos de enrutamiento pedidos en el enunciado (Dijkstra, Flooding, Link State Routing y Distance Vector Routing) sobre una arquitectura común de nodos que se comunican por sockets TCP. Cada nodo corre dos hilos en paralelo que serían forwarding y routing.

1.1 Arquitectura común

Todos los algoritmos heredan de una clase base que centraliza la parte de red: apertura del socket servidor, hilos de forwarding y de routing, envío de HELLO a los vecinos al arrancar, y el despacho de paquetes entrantes según su tipo. Cada algoritmo solo sobrescribe lo que le es propio: cómo construye su tabla de ruteo y cómo procesa los paquetes de tipo info.

El formato de los mensajes sigue la estructura JSON del enunciado, con un campo adicional "id" para poder deduplicar paquetes en Flooding y en la difusión de LSPs de Link State Routing.

1.2 Dijkstra (estático)

Recibe la topología completa al arrancar y calcula una sola vez su tabla de ruteo con el algoritmo de Dijkstra clásico. El motor de Dijkstra se implementó en un módulo aparte, sin ninguna dependencia de sockets ni de mensajes, para poder reutilizarlo también dentro de Link State Routing, cumpliendo el requisito de modularidad del enunciado. A partir del árbol de caminos más cortos se deriva la tabla next-hop, para cada destino se retrocede por los predecesores hasta encontrar el vecino directo del nodo origen.

1.3 Flooding

Cada nodo solo conoce a sus vecinos directos. Al recibir un paquete de datos que no es para él, lo reenvía a todos sus vecinos excepto por donde llegó, usando el id del mensaje para no reenviarlo más de una vez y un TTL que se decrementa en cada salto para descartar paquetes que ya dieron muchas vueltas. Flooding no construye una tabla de ruteo por lo que su tabla queda intencionalmente vacía; la entrega se confirma por la recepción del mensaje en el destino.

1.4 Link State Routing (LSR)

Cada nodo empaqueta el costo de sus enlaces directos en un Link State Packet (LSP) y lo difunde a toda la red reutilizando la misma lógica de Flooding. Cada nodo va armando su propia

base de datos de topología a partir de los LSPs recibidos de todos los demás nodos, y con esa topología derivada corre el mismo motor de Dijkstra del punto 1.2 para recalculer su tabla de ruteo cada vez que llega un LSP nuevo.

1.5 Distance Vector Routing (DVR)

Cada nodo mantiene su propio vector de distancias (destino → costo) y lo envía periódicamente a cada vecino directo, aplicando split horizon: no le anuncia a un vecino las rutas cuyo siguiente salto es precisamente ese vecino, para reducir loops y el problema de count-to-infinity. Al recibir el vector de un vecino, el nodo recalcula por completo su vector combinando todos los vectores de vecinos conocidos más el costo del enlace hacia cada uno (Bellman-Ford), lo que le permite reflejar tanto mejoras como empeoramientos de ruta en cuanto llega una actualización.

1.6 Topología de prueba

Todas las pruebas se corrieron sobre la topología de 9 nodos (A–I) del Anexo del enunciado, usando los archivos de ejemplo topo-ejemplo.txt y pesos-ejemplo.txt:

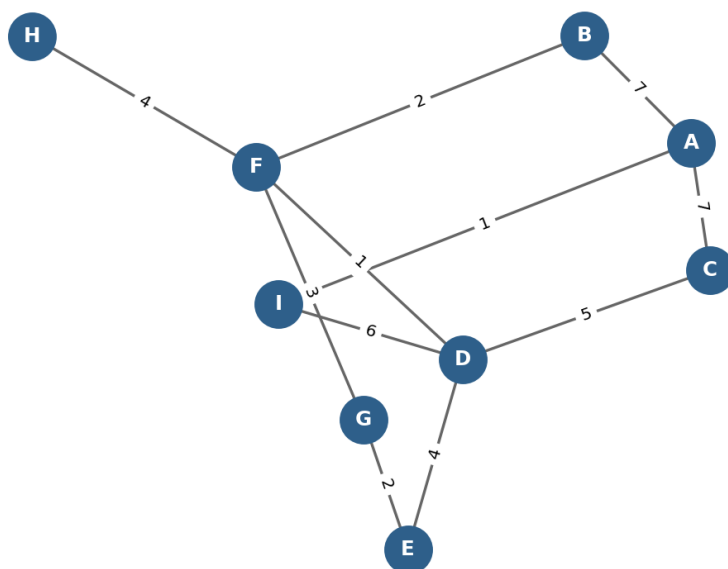


Diagrama generado a partir de los propios archivos de configuración de prueba.

2. Resultados

Se levantaron los 9 nodos por sockets TCP locales para cada uno de los 4 algoritmos, usando la topología de la sección 1.6, y se envió un mensaje de usuario de A hacia H en cada caso.

2.1 Entrega de mensajes y tiempo de convergencia

Algoritmo	Mensaje A→H entregado	Tiempo hasta tabla estable
Dijkstra	Sí	Inmediato (calculada una sola vez al arrancar)
Flooding	Sí (por inundación, sin tabla)	No aplica

Link State Routing	Sí	~10 s (con hello_interval=2s en esta prueba)
Distance Vector Routing	Sí	~10 s (con interval=2s en esta prueba)

2.2 Tabla de ruteo resultante — Dijkstra

Válida como referencia: es la solución óptima exacta del grafo completo.

Nodo	A	B	C	D	E	F	G	H	I
A	—	B	C	I	I	I	I	I	I
B	A	—	F	F	F	F	F	F	A
C	A	D	—	D	D	D	D	D	A
D	I	F	C	—	E	F	F	F	I
E	D	G	D	D	—	G	G	G	D
F	D	B	D	D	D	—	G	H	D
G	F	F	F	F	E	F	—	F	F
H	F	F	F	F	F	F	F	—	F
I	A	A	A	D	D	D	D	D	—

2.3 Tabla de ruteo resultante — Link State Routing

Calculada dinámicamente por cada nodo a partir únicamente de los LSPs recibidos por flooding.

Nodo	A	B	C	D	E	F	G	H	I
A	—	B	C	I	I	I	I	I	I
B	A	—	F	F	F	F	F	F	A
C	A	D	—	D	D	D	D	D	A
D	I	F	C	—	E	F	F	F	I
E	D	G	D	D	—	G	G	G	D
F	D	B	D	D	D	—	G	H	D
G	F	F	F	F	E	F	—	F	F
H	F	F	F	F	F	F	F	—	F
I	A	A	A	D	D	D	D	D	—

2.4 Tabla de ruteo resultante — Distance Vector Routing

Calculada dinámicamente por cada nodo únicamente a partir de los vectores de sus vecinos directos.

Nodo	A	B	C	D	E	F	G	H	I
A	—	B	C	I	I	I	I	I	I
B	A	—	F	F	F	F	F	F	A
C	A	D	—	D	D	D	D	D	A
D	I	F	C	—	E	F	F	F	I
E	D	G	D	D	—	G	G	G	D
F	D	B	D	D	G	—	G	H	D
G	F	F	F	F	E	F	—	F	F
H	F	F	F	F	F	F	F	—	F
I	A	A	A	D	D	D	D	D	—

2.5 Ruta seguida por el mensaje de prueba (A → H)

En los tres algoritmos que producen tabla de ruteo, el mensaje de A hacia H siguió exactamente la misma ruta:

A → I → D → F → H (costo total: 1 + 6 + 1 + 4 = 12)

En Flooding no hay una única "ruta": el mensaje llega a H por inundación, potencialmente habiendo recorrido varias copias por distintos caminos en paralelo, hasta que la primera copia llega y las demás se descartan por deduplicación.

3. Discusión

Los resultados obtenidos muestran una alta consistencia entre los algoritmos implementados. Dijkstra y Link State Routing (LSR) produjeron exactamente la misma tabla de ruteo en los nueve nodos de la red. Este comportamiento era el esperado, ya que LSR termina resolviendo el mismo problema de camino más corto que Dijkstra, con la diferencia de que lo hace de forma distribuida. Mientras Dijkstra recibe la topología completa desde un archivo de configuración, en LSR cada nodo construye su propia representación de la red a partir de los paquetes LSP recibidos. Por lo tanto, la coincidencia exacta entre sus tablas de ruteo representa una validación importante de la implementación de LSR y, en particular, de la reutilización del algoritmo de Dijkstra para calcular las rutas de menor costo.

En el caso de Distance Vector Routing (DVR), los resultados coincidieron con la referencia de Dijkstra en ocho de los nueve nodos. La única diferencia se presentó en el nodo F para alcanzar el destino E. DVR seleccionó a G como siguiente salto, obteniendo un costo total de 5 mediante la ruta F→G→E (3 + 2), mientras que Dijkstra seleccionó a D, también con un costo total de 5 mediante F→D→E (1 + 4). Debido a que ambas rutas poseen exactamente el mismo costo, esta diferencia no representa un error en DVR, sino un empate entre dos caminos igualmente óptimos. La elección final depende de la forma y el orden en que se procesan las actualizaciones de los vectores de distancia. Este resultado evidencia una característica importante de DVR: cuando existen varios caminos con el mismo costo mínimo, el algoritmo no necesariamente selecciona la misma ruta que Dijkstra, aunque sí encuentra una ruta óptima.

También se observaron diferencias en la velocidad de convergencia de los algoritmos. Dijkstra fue el más rápido en disponer de una tabla de ruteo completa, debido a que no depende del intercambio de información entre nodos y conoce la topología desde el inicio. Flooding, por su parte, no requiere calcular ni mantener una tabla de ruteo, por lo que no posee un tiempo de convergencia comparable con los demás algoritmos. En contraste, LSR y DVR necesitan varias rondas de intercambio de información para que el conocimiento de la red alcance a los nodos más alejados. En esta prueba, utilizando un intervalo de anuncio de 2 segundos, ambos algoritmos tardaron aproximadamente 10 segundos en estabilizarse. Este comportamiento es consistente con el diámetro de la red utilizada, que alcanza hasta cuatro saltos entre los nodos más distantes, y demuestra que la convergencia depende, entre otros factores, de la distancia que debe recorrer la información dentro de la red.

Otro aspecto relevante es el costo de mantenimiento o overhead generado por cada algoritmo. Flooding no utiliza tráfico de control para mantener tablas de ruteo, ya que simplemente reenvía los mensajes de datos hacia sus vecinos; sin embargo, esto puede generar una cantidad considerable de tráfico por cada mensaje transmitido. LSR, en cambio, distribuye paquetes LSP a través de la red de manera periódica según el `hello_interval`, lo que implica un mayor uso de ancho de banda para mantener actualizada la información de topología, pero permite calcular rutas óptimas de manera consistente. DVR únicamente intercambia sus vectores de distancia con los vecinos directos, lo que reduce el alcance del tráfico de control en cada ronda. No obstante, requiere varias rondas para propagar la información de un extremo de la red al otro y puede presentar problemas como `count-to-infinity` ante cambios o fallas en los enlaces. En esta implementación, dicho problema se mitiga mediante `split horizon`, aunque las pruebas realizadas no incluyeron escenarios de fallas de nodos o enlaces.